Basic (a simple version log) — Hints

Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

Two real Python skills carry this whole exercise: writing/reading a JSON file, and building up a list of dicts one entry at a time. `json.dump(data, f, indent=2)` writes readable JSON; `indent=2` is optional but makes the file human-readable if you ever open it directly. `json.load(f)` reads it back into the exact same Python structure (list of dicts) you saved.

For the timestamp, `datetime.now(timezone.utc).isoformat()` gives you a sortable, unambiguous string like `"2026-09-11T14:03:00+00:00"` — always store it as UTC, not local time, so it means the same thing no matter who reads the log later.

The exact pieces:

- `import json` — `json.dump(data, f, indent=2)` and `json.load(f)`.
- `from datetime import datetime, timezone` — `datetime.now(timezone.utc).isoformat()`.
- `pathlib.Path(path).exists()` — check whether the log file already exists before deciding whether to start a new list or load the existing one.
- A stand-in for Doc13's test suite: a small function, `run_eval_suite(prompt_text)`, that runs a handful of test cases against the prompt and returns a score. In this exercise you can stub it (fake, deterministic answers) so the script runs on its own — in real Project 5 code, this calls your actual Doc13 suite.

Hint 2 — The plan, and almost the whole thing

```
function save_version(name, prompt_text, path) -> None:
    load existing list from path, or start a new one if the file doesn't exist
    append {"version_name": name, "prompt_text": prompt_text,
            "created_at": utc now, isoformat}
```

```
    write the whole list back to path

function load_versions(path) -> list[dict]:
    if file doesn't exist: return empty list
    read and return the JSON list

save_version("v1", ..., "versions.json")
save_version("v2", ..., "versions.json")

for version in load_versions("versions.json"):
    run the eval suite, print name and score
```

```
# version_log_practice.py
import json
from datetime import datetime, timezone
from pathlib import Path

def save_version(name: str, prompt_text: str, path: str) -> None:
    versions = load_versions(path)
    versions.append({
        "version_name": name,
        "prompt_text": prompt_text,
        "created_at": datetime.now(timezone.utc).isoformat(),
    })
    with open(path, "w") as f:
        json.dump(versions, f, indent=2)

def load_versions(path: str) -> list[dict]:
    if not Path(path).exists():
        return []
    with open(path) as f:
        return json.load(f)
```

Now write the loop that calls `run_eval_suite` for each saved version and prints its score, and compare against the Solution.

Full solution: Show me the solution